VOLUME 04 / 08

HAZARDS AND CONTROL

Forward available values, wait for loads and discard wrong paths

Implement operand forwarding, WB-to-ID bypass, load interlocks, redirects and halt/drain behavior.

FIG 06 / EX OPERAND SELECTION

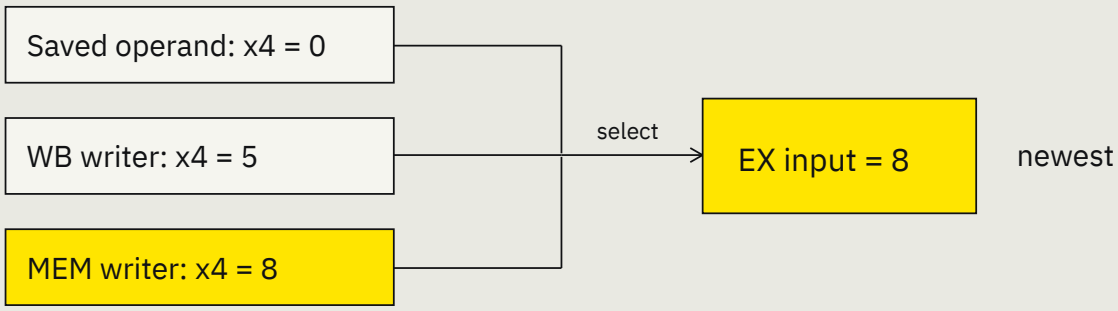


Illustration: the matching non-load MEM result has priority over WB.

A hands-on companion to the RISC-V project. Examples, implementation steps, debugging and completion checks. Personal design edition / October 2026.

What you will build

You will preserve architectural results when instructions depend on unfinished work or change the control path.

Before you start

Complete the stage transfers from Volume 3. Know when each result becomes available.

Files and outputs

rtl/rv32_core.sv: operand block, uses_rs*, hazard, redirect/fault and front-end edge cases. Teaching cases: forward_priority, wb_id, load_store, load_branch, load_jalr and branch_flush.

Build order

1. Build producer/consumer timing tables.
2. Implement newest-result forwarding for both sources.
3. Add same-edge WB bypass.
4. Add one-cycle load interlock.
5. Implement EX redirect and trap drain.
6. Check false dependencies and wrong-path side effects.

The checked-in core is an assistant-generated reference. Work on a learning branch. The code excerpts explain selected regions and are not complete replacement modules unless explicitly labeled.

Forwarding selects the newest available value

Each EX input can come from its saved ID operand, a WB result, or a newer non-load result in MEM. The hardware compares destination and source register numbers.

```
value = saved_operand
if WB is a valid matching writer: value = WB.result
if MEM is a valid matching non-load writer:
    value = MEM.result
```

The last matching assignment wins in this combinational calculation. MEM represents a younger producer than WB, so its value has priority. x0 never participates as a producer.

> **WORKED EXAMPLE**
>
> If an older instruction writes x1=5 and a newer instruction writes x1=8, a following consumer must use 8. Keeping the WB value when a newer MEM result matches would be wrong.

The same source selection feeds store data, branch comparisons, JALR bases and DOT4. Checking only ordinary ADD operands leaves important hazards unresolved.

FIG 06 / EX OPERAND SELECTION

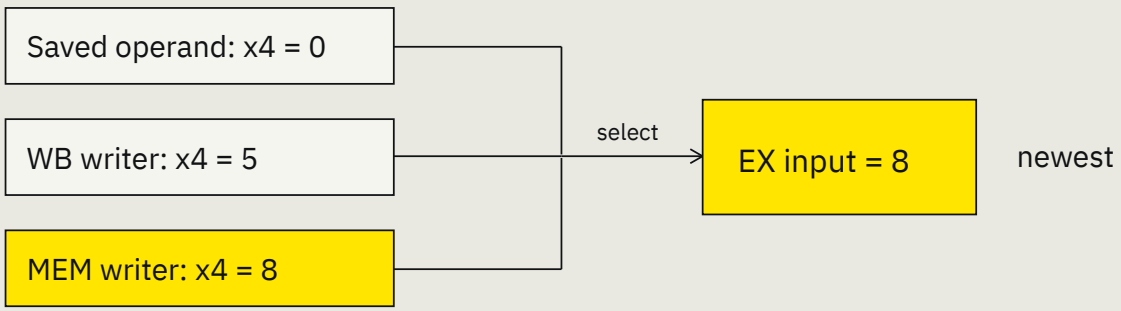


Illustration: the matching non-load MEM result has priority over WB.

Writeback and decode can meet on one edge

The register file writes at a rising edge. ID/EX also captures its inputs at that edge. With nonblocking assignments, a register-file read expression can still see the old value during evaluation.

```
addi x1, x0, 7
addi x8, x0, 1
addi x9, x0, 2
add  x2, x1, x1
```

When the ADD reaches ID, the first ADDI is in WB. The core explicitly bypasses the WB value into the captured ID operand if the source register matches.

This **WB-to-ID bypass** solves a different timing boundary from the EX operand forwarding muxes. Both can matter in the same implementation.

A good test varies dependency distance. Adjacent instructions exercise one path. Two intervening instructions can exercise another. Also vary whether the dependency appears on rs1, rs2 or both.

For a store, distinguish the address base from the stored value. They are separate dependencies even though both use the same register file.

Clock interval	Producer	Consumer	Needed path
Same interval	WB writes x1	ID captures x1	WB to ID/EX bypass

A load address is not the loaded value

```
lw  x1, 0(x0)
add x2, x1, x1
```

The load calculates its address in EX. Its data becomes available during MEM and is captured into MEM/WB. When the following ADD would enter EX, that loaded value is not yet available through this core's forwarding paths.

The interlock holds PC and IF/ID for one cycle, inserts an invalid EX entry and lets the older load advance. On the next cycle, the ADD enters EX and receives the loaded value from WB.

The hazard detector checks whether the ID instruction really uses rs1 or rs2. Bits in an immediate can occupy the rs2 field position without naming a source register. Treating those bits as a dependency causes unnecessary stalls.

This is a one-cycle stall only under the chosen memory and forwarding contract. A slower memory needs a different protocol and possibly more waiting.

At stall edge	Action
PC	Hold current fetch address
IF/ID	Hold consumer instruction
ID/EX	Insert invalid bubble
EX/MEM and MEM/WB	Older work continues

Step 1: wire both operand muxes

Use ID/EX values as defaults. Override them with matching valid WB results. Finally override with a matching non-load MEM result, which represents the newer producer.

```
a = x_a; b = x_b;
if (w_valid && w_write && w_rd != 0) begin
    if (rs1 == w_rd) a = w_result;
    if (rs2 == w_rd) b = w_result;
end
if (m_valid && m_write && !m_load && !m_trap && m_rd != 0) begin
    if (rs1 == m_rd) a = m_result;
    if (rs2 == m_rd) b = m_result;
end
```

This is the reference's combinational ordering. Ensure trap handling suppresses writes. Loads are excluded from the MEM bypass because their m_result contains the address.

Feed a and b into branch comparisons, store-data selection, JALR and DOT4 as well as ALU arithmetic. Testing only an ADD chain leaves those consumers unproven.

> **TRY IT BEFORE READING THE ANSWERS**
>
> 4A. Two consecutive instructions write x1=5 and x1=8. What value must a following ADD use?

Step 2: handle the WB/ID coincidence

When an ID instruction captures operands on the same edge that WB writes the register file, an ordinary array read can still see old contents. Explicitly select the old w_result for a matching source.

```
x_a <= f_insn[19:15] == 0 ? 32'b0 :
    (w_valid && w_write && !w_trap &&
     w_rd == f_insn[19:15] ? w_result : regs[f_insn[19:15]]);
```

Mirror the expression for rs2. The source register here comes from f_insn, because the consumer is in ID. The EX bypass comparisons instead use the instruction already in x_insn.

Use the `wb_id` case: producer, two unrelated instructions, then consumer. Adjacent dependencies alone do not isolate this boundary.

If you deliberately remove this bypass, predict which operand the consumer captures. In a temporary experiment, verify the first failing result and restore the good design.

Step 3: identify a true load consumer

The producer is the valid load in EX. The consumer is the valid instruction in ID. Check a nonzero load destination against each source that the consumer actually uses.

```
wire hazard = f_valid && x_valid && ld && !trap &&
    x_insn[11:7] != 0 &&
    ((uses_rs1 && f_insn[19:15] == x_insn[11:7]) ||
     (uses_rs2 && f_insn[24:20] == x_insn[11:7]));
```

Construct uses_rs1 and uses_rs2 from the consumer's opcode. ADDI uses rs1 and an immediate. Its bits 24:20 are not an rs2 dependency. Stores use both a base and data source. DOT4 uses both sources.

For `lw x0, ...`, no consumer needs its discarded result. A false stall is a performance defect even when the final values are correct.

> **TRY IT BEFORE READING THE ANSWERS**
>
> 4B. Why can comparing every instruction’s bits 24:20 create a false hazard?

Step 4: insert one bubble without stopping the load

At the interlock edge, hold PC and IF/ID. Clear x_valid so EX receives a bubble. Continue the previously described EX/MEM and MEM/WB updates.

```
if (redirect || fault) begin
    f_valid <= 0; x_valid <= 0;
    if (redirect) pc <= target;
    if (fault) halted <= 1;
end else if (hazard) begin
    x_valid <= 0;
end else begin
    // Ordinary ID/EX and IF/ID capture, then PC advance.
end
```

Leaving a register unassigned in this clocked branch holds its old value. That is intended storage, unlike an omitted assignment in a combinational block.

The older producer advances independently. In the next interval, the load reaches WB and the consumer can enter EX with a forwarded result.

A redirect/fault outranks the younger dependency. Do not freeze a wrong-path instruction merely because it happens to look dependent.

FIG 07 / LOAD, ADD, STORE

Instruction	1	2	3	4	5	6	7	8
lw x1, 0(x0)	IF	ID	EX	MEM	WB			
add x2, x1, x1		IF	ID	ID held	EX	MEM	WB	
sw x2, 4(x0)			IF	IF held	ID	EX	MEM	WB
EX bubble				bubble				

Illustration: one held front-end cycle creates one invalid EX entry.

A taken branch discards two younger stages

```
beq x1, x1, target
sw  x2, 128(x0)      # wrong path
addi x3, x0, 99      # wrong path
target: addi x4, x0, 42
```

The branch decides in EX. At that point, younger instructions occupy ID and IF. A taken branch invalidates both. Fetch resumes at the target on the following cycle.

With no other stalls, this design pays two lost issue opportunities for an EX redirect. An untaken branch continues sequential fetching. Other pipelines can resolve branches later and incur a different penalty.

JAL writes a return address and redirects using a PC-relative immediate. JALR adds a signed immediate to a register base and clears target bit zero. This core traps if the resulting taken target is not word aligned.

Forwarded operands must feed branch comparisons and JALR bases. Otherwise a correct arithmetic pipeline can still choose the wrong instruction path.

FIG 08 / EX-STAGE REDIRECT

Instruction	1	2	3	4	5	6
branch	IF	ID	EX	MEM	WB	
wrong-path store		IF	ID / discard			
wrong-path ADDI			IF / discard			
target				IF	ID	EX

Illustration: the branch discards younger IF and ID instructions at the end of cycle 3.

Step 5: squash effects, then drain traps

EX decides a redirect. At its edge, clear the younger f_valid and x_valid entries. The target becomes the IF address in the next interval. Do not capture the old wrong-path instruction response as target work.

A trap similarly invalidates younger work and sets halted. Older instructions continue toward WB. The faulting instruction carries a trap record while its register/memory controls are disabled.

Use `branch_flush` to check a skipped store, `load_branch` to exercise a loaded comparison operand, and `load_jalr` to exercise a loaded target base.

Observe memory writes separately. Correct final register values can coexist with an incorrect transient store. Place distinctive data at the wrong-path destination and verify that no write event occurs there.

The physical memory side effect occurs in MEM. Valid bits and squash timing must prevent wrong-path stores from reaching that stage as valid stores.

Step 6: make a hazard test matrix

Cross producer type with consumer use, source position and separation. At minimum, test ALU and load producers feeding ALU, store address, store data, branch, JALR and DOT4.

Also test competing writers, x0, a dependency on both sources, a same-edge WB/ID case and nonconsumer immediate bits. Verify exact stall behavior with a dedicated timing check when the claim concerns performance.

The learning runner prints stall and flush event counts for each small program. These events are sampled by the current testbench on falling edges. Align the definition before treating them as a general CPI measurement.

To debug, find the consumer's EX cycle, identify the selected source, then ask where the newest producer resides. If that source does not yet exist, inspect the interlock. If the consumer is wrong-path work, inspect flush validity instead.

> **TRY IT BEFORE READING THE ANSWERS**
>
> 4C. If a load-to-store program stores the load address, which rule was probably violated?

A research experiment for this part

Write this entry before editing. It is a proposed experiment, not a result already obtained by the learner.

Question	Does the newest available producer always reach the consumer?
Prediction	A matching non-load MEM result overrides an older WB result.
Experiment	Run competing writers; disable the priority in an isolated experiment and identify the first mismatch.
Evidence	Small program, failure trace, patch and passing result after restoration.

What to record

Save source revision, tool versions, input words, seed, exact command, expected and observed behavior, raw artifact path and limitation. End with the decision that the observation supports.

Use `docs/EXPERIMENT_LOG.md`. If the result contradicts the prediction, keep the failure and explain the next test. Do not rewrite the prediction afterward to make the experiment appear successful.

Answers and completion check

4A. Use 8. The newer matching MEM producer must override the older WB producer.

4B. Immediate bits can resemble a register index without naming a register source. Decode actual source use before comparing.

4C. A load address was probably forwarded as data, or the load-use bubble was missing. MEM-load producers must not use the non-load result bypass.

Completion check

All named teaching hazards pass, no wrong-path store appears, and you can draw the load/add/store and EX branch timelines. The reference's generated hazard cross (`tests/hazards.py`) is the next step up.

Sources and next steps

These sources support the concepts in this volume. The current RTL and local execution contract remain the authority for implementation-specific behavior.

1. Brown pipeline notes
2. RISC-V ISA
3. Project source

Detailed chapter-mapped reading is in `docs/textbook/SOURCES.md`. Research-interest context is in `docs/RESEARCH.md`. The original overview and its interactive figures remain companion material.

Keep the evidence boundary

Teaching labs are separate from the 253-program baseline. Reference coverage and timing numbers belong to the recorded source/tool configuration. No physical board execution, ISA certification or formal proof is implied. The DOT4 speedup covers one kernel.

Continue with Volume 5. Run the complete regression before treating a learning edit as a finished implementation.